SABUDH

PROGRAMMING ASSIGNMENT

Objective:

The objective of this assignment is to assess the student's understanding of object-oriented programming concepts and their ability to apply these concepts using Python.

Instructions:

1. Submit a single Python file (.py) containing the complete solution.
2. The code must run without modification.
3. The program must not use any third-party libraries. Only built-in Python modules are allowed.
4. Use clear variable names, proper indentation, and comments explaining important parts of the code.
5. All classes must follow cooperative inheritance using super().
6. All attributes and methods must use exact names as specified to allow automated grading.
7. The final file must contain a main execution block: if __name__ == "__main__":
   This block should demonstrate the functionality of the classes.
8. Do not hardcode class names inside methods where dynamic class names are required. Use self.__class__.__name__.

Task 1: Abstraction – Create an Abstract Class

Create an abstract class called **Person** using the abc module. The class must inherit from ABC.

The class must contain the following attributes:

- Name
- age
- gender
- address

All attributes must be initialized using a constructor.
Use cooperative inheritance by calling super().__init__() where applicable.

Methods

1. Magic Method

Implement the __str__() method that returns the following format:

Name: <name>, Age: <age>, Gender: <gender>, Address: <address>

2. greet Method

Create a method: greet(self, other_person)

Where other_person is an instance of the Person class.

The output should be: Hello <other_person.name>! My name is <self.name>.

3. Abstract Method

Define an abstract method called: introduce(self)

This method must be implemented in all child classes. Use the @abstractmethod decorator.

4. Static Method

Create a static method: is_adult(age)

The method must return:

- True if age ≥ 18
- False otherwise

Task 2: Single Inheritance and Encapsulation

Create a Python class called **Employee** that inherits from the **Person** class created in **Task** 1.

The class must implement cooperative inheritance, meaning the constructor must call the parent constructor using super().__init__().

Attributes

The **Employee** class must contain the following attributes.

1. Class Attribute

Create a class attribute called: counter

It should track the **number of Employee objects currently created**.

The counter should:

- **Increase by 1** when a new Employee object is created.
- **Decrease by 1** when an Employee object is deleted.

2. Private Attribute

Create a **private attribute**:__employee_id

The employee ID must be automatically generated using the counter in the following format:

EMP01
EMP02
EMP03

Rules:

- The employee ID **must be assigned automatically during object creation**.
- The attribute must be **private**.
- Only a **getter method** should be provided.
- The employee ID **cannot be modified after creation** (no setter allowed).

3. Protected Attribute

Create a **protected attribute**:_salary

This attribute should store the salary of the employee.

Methods

1. Constructor

Implement the constructor to:

- Call the **parent constructor using super()**
- Initialize _salary
- Automatically generate the __employee_id
- Increment the class variable counter

2. Destructor

Implement a destructor:__del__()

This method should **decrease the class variable counter when an Employee object is deleted**.

3. Counter Property

Create a **property method** called: employee_count

This method should **return the current value of the class variable counter**.

4. Employee ID Getter

Create a **getter method** that returns the private attribute __employee_id.

No setter method should be defined.

5. Salary Methods

Implement the following methods:

- **Getter for salary**
- **Setter for salary**
- A method to **increase the salary**
- A method to **decrease the salary**

6. introduce Method

Override the abstract method defined in the **Person** class.

Example output: Hello, my name is John and I work as an employee.

(The exact wording can vary, but it must clearly introduce the employee.)

Task 3: Multiple Inheritance and Polymorphism

Create a Python class called **Teacher** that inherits from the **Employee** class.

The class must implement **cooperative inheritance**, meaning the constructor must call the parent constructor using super().__init__().

Attributes

1. Class Attribute

Create a class attribute called: counter

This attribute should track the **number of Teacher objects currently created**.

Rules:

- Increase the counter by **1** when a new Teacher object is created.
- Decrease the counter by **1** when a Teacher object is deleted.

2. Private Attribute

Create a **private attribute**:__teacher_id

The ID should be automatically generated using the counter in the format:

TEC01
TEC02
TEC03

Rules:

- It must be assigned **automatically during object creation.**
- The attribute must be **private**.
- Provide **only a getter method**.
- The value **must not be modified after creation**.

3. Subjects Attribute

Create an attribute:subjects

This should store a **list of subjects taught by the teacher**.

Methods

1. Constructor

The constructor must:

- Call the parent constructor using super()
- Initialize the subjects list
- Automatically generate the __teacher_id
- Increase the class variable counter

2. Destructor

Implement:__del__()

This method should **decrease the class variable counter when a Teacher object is deleted**.

3. Teacher Count Property

Create a property method called:teacher_count

This should return the current value of the class variable counter.

4. Teacher ID Getter

Create a **getter method** for the private attribute __teacher_id.

No setter method should be implemented.

5. Subject Management Methods

Create methods to:

- **Add a subject** to the subjects list
- **Remove a subject** from the subjects list

6. Method Overriding (Polymorphism)

Override the introduce() method defined in the **Person** class.

The method should return:

- the teacher_id
- the list of subjects taught

Example output: My ID is TEC01 and I teach: Mathematics, Physics

7. Override employee_id

Since teachers have their own identifier (teacher_id), the attribute employee_id should **not be accessible**.

Override the employee_id property so that accessing it raises an error:

AttributeError: <ClassName> object has no attribute 'employee_id'

8. Dynamic Class Name Requirement

When raising the error in the previous step, the **class name must not be hardcoded**.

Instead, use: self.__class__.__name__

This ensures that if another class inherits from **Teacher**, the error message automatically reflects the correct class name.